

XRPL Validator — Daily Engineering Report • 2026-05-02

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- 3 commits (xrpl-validator 3)

Engineering changes

- **fix(ffi): suppress no-op Modifieds from mutation-divergence comparison** (a43897b , xrpl-validator) Rippled's meta-build at ApplyStateTable.cpp:155-156 skips Modified items where the new SLE bytes equal the pre-state bytes (a "touched but not changed" no-op). The OUTER apply path (line 47-48) calls to.rawReplace(sle) for all modify items unconditionally, so libxrpl's MutationCollector records every touch — including the no-ops that the network meta filtered out.
- **fix(ffi): use closed-ledger OpenView so meta-build runs** (4242a8e , xrpl-validator) ApplyStateTable::apply() at line 127 gates the meta-build branch on `if (!to.open() || isDryRun)` . Inside that branch, threadItem and threadOwners run — these add threading-only mutations (PreviousTxnID bumps on related accounts: NFT issuer for transfer-fee NFTs, trustline counterparty for TrustSet, etc.) to the items_ set.
- **fix(ffi): set header.seq = ledger_seq (not -1) for closed-ledger view** (0c80324 , xrpl-validator) Yesterday's switch from open_ledger to closed-ledger OpenView mode inverted the meaning of header.seq. Open-ledger ctor does `info_.seq = base.seq + 1` , so passing `header.seq = ledger_seq - 1` yielded `view.seq() = ledger_seq` . Closed-ledger ctor copies info_ as-is — no +1 — so passing `header.seq = ledger_seq - 1` yields `view.seq() = ledger_seq - 1` , off by one.

Appendix — commits

Time	Repo	Commit	Subject
07:24	xrpl-validator	a43897b	fix(ffi): suppress no-op Modifieds from mutation-divergence comparison
07:40	xrpl-validator	4242a8e	fix(ffi): use closed-ledger OpenView so meta-build runs
19:29	xrpl-validator	0c80324	fix(ffi): set header.seq = ledger_seq (not -1) for closed-ledger view